

HW4 Solutions

Paul Schliekelman

Problem 1

First, generate some data:

```
##generate data:
library(MASS)
set.seed(1234)

#sample sizes for the three categories 1, 2, and 3:
n=50
n1=n
n2=n
n3=n

#means for the three categories
mu_x1=10
mu_y1=10
mu_x2=30
mu_y2=30
mu_x3=10
mu_y3=30

sd0=60 #SD

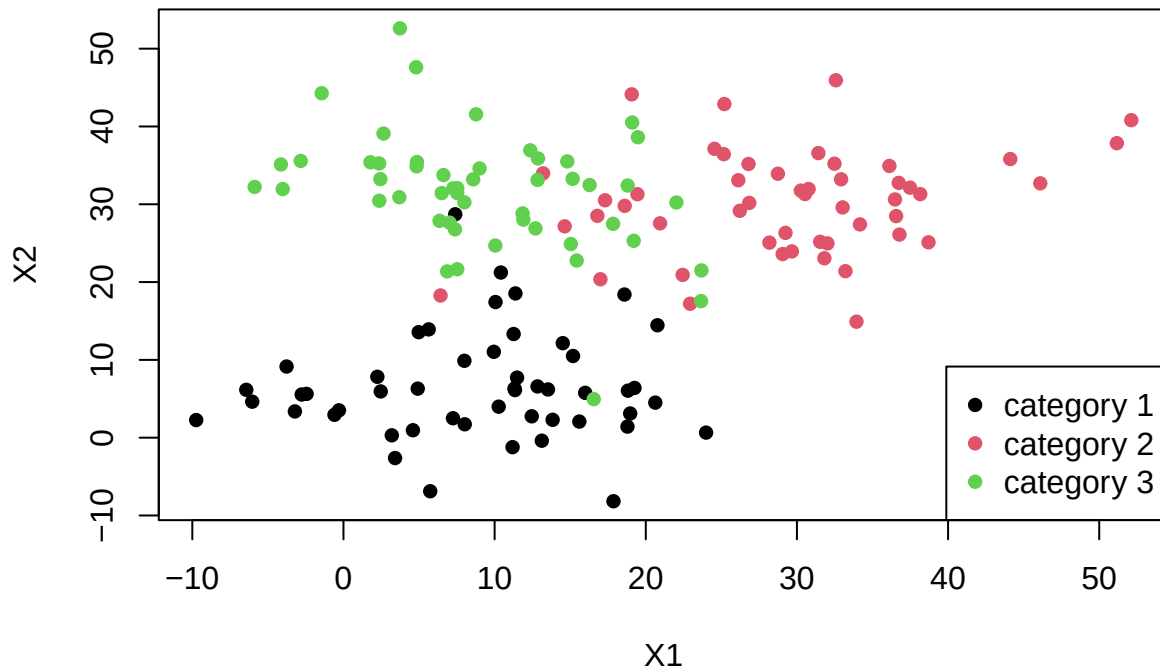
#generate values for sample 1:

d1=mvrnorm(n1,mu=c(mu_x1,mu_y1),Sigma=sd0*diag(2))
d2=mvrnorm(n2,mu=c(mu_x2,mu_y2),Sigma=sd0*diag(2))
d3=mvrnorm(n3,mu=c(mu_x3,mu_y3),Sigma=sd0*diag(2))

dd=rbind(d1,d2,d3)
dd=cbind(dd,c(rep(1,n1),rep(2,n2),rep(3,n3)))

plot(dd[,1:2],col=dd[,3],xlab="X1",ylab="X2",main="Sample Data",pch=16)
legend("bottomright",c("category 1","category 2","category 3"),col=1:3,pch=16)
```

Sample Data



Here is my function. Explanations are given in comments:

```
library(class)

#function to find the optimal value of K in k-nearest neighbors algorithm
#datX is training data frame with predictors
#datC has classes for training data
#nfold is the number of folds in cross-validation
#tests values of k from 1:maxk
find_k<-function(datX,datC,nfold,max_k)
{
  require(class)
  n=nrow(datX) #sample size
  per_fold=n/nfold #number of data points per fold
  if(per_fold!=floor(per_fold)) stop("data does not divide evenly into folds")

  #shuffle rows of data, then divide into folds
  samp_perm=sample(1:n) #permutation of sample
  datX=datX[samp_perm,] #randomly permute data
  datC=datC[samp_perm]
  folds=rep(1:nfold,each=per_fold) #fold indexes for permuted data

  class_acc=matrix(nrow=nfold,ncol=max_k) #matrix to hold prediction accuracy for
                                           #training data

  #loop over folds and over values of k
  #perform knn using data not in fold j to predict classes for fold j
  #calculate proportion of correct prediction for each fold and k
  #note that knn algorithm resolves ties at random. This produces random variation
```

```

#in the optimal k even with nfold=150
for(j in 1:nfold)
{ for(i in 1:max_k)
  { knnout=knn(datX[folds!=j,],datX[folds==j,],datC[folds!=j],k=i)
    class_acc[j,i]=sum(knnout==datC[folds==j])/length(knnout)

  }
}

#best k is the one that produces the highest mean prediction accuracy
avg_acc=apply(class_acc,2,mean)

(best_k=(1:max_k)[which.max(avg_acc)])

return(list(best_k=best_k,accuracy=cbind(1:max_k,avg_acc)))
}

```

Test the function with LOOCV and the data set that we constructed.

```

fk=find_k(dd[,1:2],dd[,3],150,30)
fk$best_k

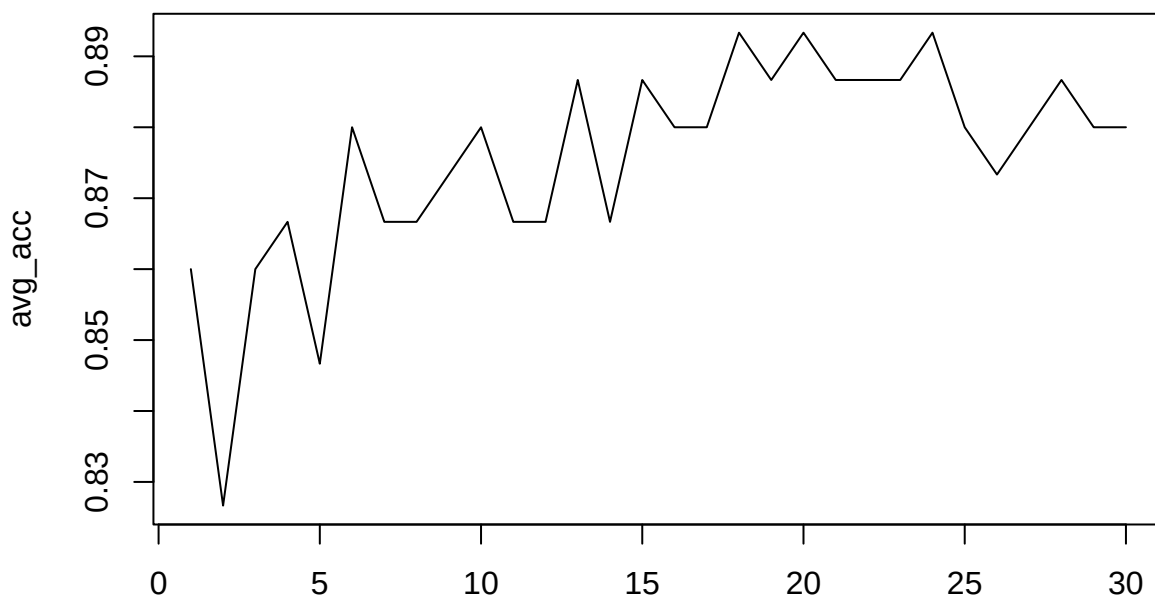
```

```
## [1] 18
```

From this function, the optimal value of k for this data is in the range 10-20 (note that if we run it multiple times we get different values). Note that for there is randomness in the optimal k resulting from two sources: First, the data points are randomly sorted into folds. This will make the data points used to predict other data points vary from run to run. Second, if there is a tie in the vote, the tie is broken at random. Thus, even if the folds are the same (as happens in LOOCV), there will still be random variation in the assigned class and therefore (potentially) in the optimal value of k .

The accuracy doesn't vary much with k :

```
plot(fk$accuracy,type="l")
```



Problem 2

First, I will make a function to generate the data. This generates two classes. For each class, the predictors are generated from a mixture of two normal distributions. The parameter `m1` specifies the number of informative predictors (i.e. predictors that vary between classes) and `m0` specify the number of uninformative predictors.

`n1` and `n2` are vectors specifying the sample sizes in the two groups within each class. The groups correspond to the components of the mixture distribution.

`X11` and `X12` specify the vector of means for the two components of class 1 . `X21` and `X22` specify the vector of means for the two components of class 2 .

`sd0` is the standard deviation of the noise component for each predictor.

The function returns the data set. If `plotY=TRUE`, then it produces a two-dimensional plot to show how the different mixture components are distributed.

```
library(MASS)

generateClassificationData<-function(n1,n2,m1,m0,x11,x12,x21,x22,sd0,plotY=TRUE,returnDAT=T)
{
  #means for each class and center:
  mean11=c(rep(x11[1],m1/2),rep(x11[2],m1/2)) #mean for class 1, center 1
  mean12=c(rep(x12[1],m1/2),rep(x12[2],m1/2)) #class 1, center 2
  mean21=c(rep(x21[1],m1/2),rep(x21[2],m1/2)) #class 2, center 1
  mean22=c(rep(x22[1],m1/2),rep(x22[2],m1/2)) #class 2 center 2

  #create data for class 1: first n1[1] are from center 1, next n1[2] are from center 2
  dat_out=rbind(mvrnorm(n1[1],mean11,diag(m1)*sd0),mvrnorm(n1[2],mean12,diag(m1)*sd0))

  #create data for class 2:
  dat_out=rbind(dat_out,mvrnorm(n2[1],mean21,diag(m1)*sd0),mvrnorm(n2[2],mean22,diag(m1)*sd0))

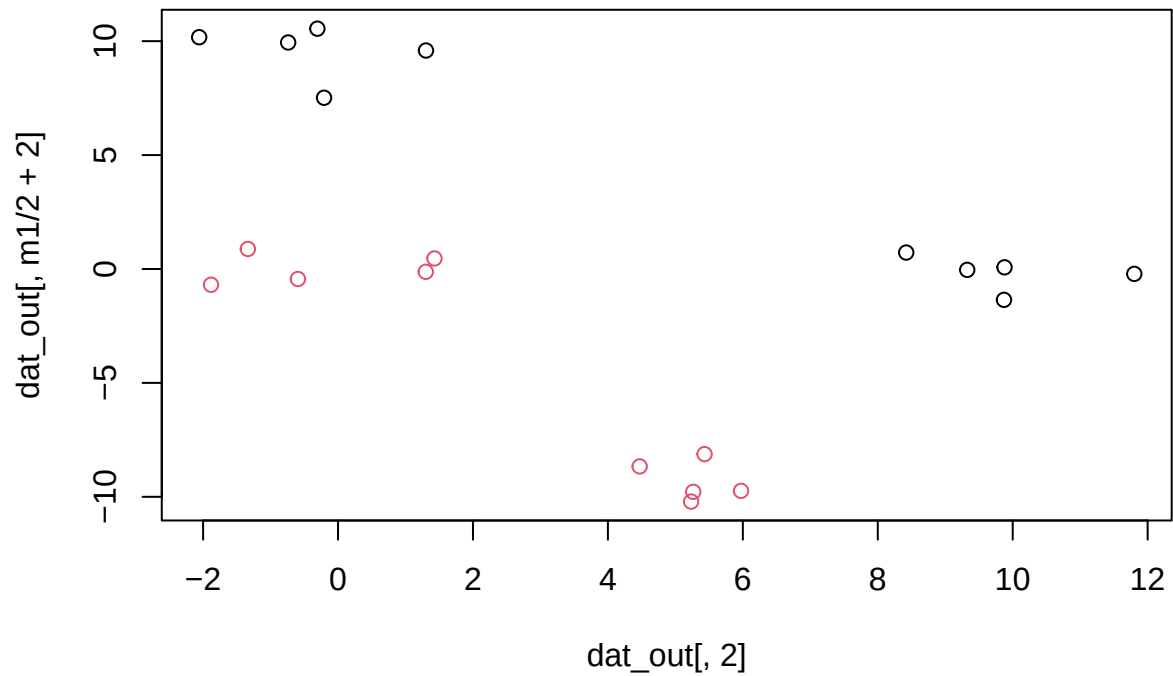
  #add noise data:
  if(m0>0) dat_out=cbind(dat_out,matrix(rnorm(m0*(sum(n1)+sum(n2)),sd=sd0),ncol=m0))

  #add class variable in first column
  dat_out=data.frame(as.factor(c(rep(1,sum(n1)),rep(2,sum(n2))))),dat_out)
  names(dat_out)=paste("v",1:ncol(dat_out),sep="")

  if(plotY==TRUE)
  {
    par(mfrow=c(1,1))
    plot(dat_out[,2],dat_out[,m1/2+2],col=dat_out[,1])
  }

  if(returnDAT==TRUE) return(dat_out)
}

#test
generateClassificationData(c(5,5),c(5,5),10,0,x11=c(10,0),x12=c(0,10),x21=c(0,0),x22=c(5,-10),1)
```



##	v1	v2	v3	v4	v5	v6	v7
## 1	1	9.8708651	9.7610560	9.6367750	11.0563764	9.08441507	-1.35467498
## 2	1	9.8784731	11.4498347	9.7990095	11.7920543	9.87156146	0.07333886
## 3	1	9.3257267	11.8528739	10.4512819	11.7108192	11.84021049	-0.03898853
## 4	1	8.4215662	9.5967685	9.5200580	9.5685711	10.45053126	0.72186988
## 5	1	11.8020764	10.8269683	8.8344748	9.9207231	10.27479368	-0.21491896
## 6	1	-0.2070870	-1.2211614	0.4472702	0.1809075	2.40240516	7.51658064
## 7	1	-2.0584127	0.8020219	-1.0440419	-0.1696028	-0.14422094	10.17283428
## 8	1	-0.7391233	0.1268307	-0.1872856	-0.2345280	0.02339644	9.94109585
## 9	1	-0.3066221	0.3885149	1.2872909	-1.3549307	1.12412194	10.54745230
## 10	1	1.3034176	-0.8721962	1.2740928	0.6299561	0.69777449	9.58952640
## 11	2	1.2999966	-1.1202125	-1.0987784	0.6211903	-0.10445279	-0.12470270
## 12	2	-1.3367868	0.3982595	1.7777421	0.6625617	-0.20799991	0.87755233
## 13	2	1.4292424	0.4213068	0.3169444	-0.6509902	1.21930841	0.46334691
## 14	2	-0.5946931	-0.4251536	1.0495764	-0.2087371	-2.27043570	-0.43936534
## 15	2	-1.8826865	-0.8769910	-0.5262808	1.7969116	-0.97345267	-0.69241729
## 16	2	5.2339183	5.2973101	5.5131208	4.8892650	5.24774665	-10.20899887
## 17	2	5.2644419	5.5574165	4.5540432	6.1921946	3.62421514	-9.78154001
## 18	2	5.4324199	3.7266630	3.1608062	3.3441141	3.80946022	-8.12652479
## 19	2	5.9732222	6.0250726	4.1801524	3.9543567	4.02618138	-9.73827820
## 20	2	4.4708059	5.4666122	4.0047905	3.2597609	4.90036880	-8.66530548
##		v8	v9	v10	v11		
## 1		0.1146617	-0.9675148	-1.6628550	-1.84347362		
## 2		1.0957404	0.6323894	0.2947297	0.60120303		
## 3		-0.4027099	0.1830787	1.5699014	-2.31793757		
## 4		-0.3590697	1.3501161	0.5119469	-0.38031207		
## 5		-0.5527882	2.4873277	1.2859024	1.65516490		
## 6		10.3497227	11.4749391	10.1925089	10.80111681		
## 7		9.3223554	11.7094721	10.8492280	8.20773527		
## 8		10.6421827	10.2283596	10.9905146	9.65258127		
## 9		8.7105620	9.7918831	10.6275822	11.55431593		
## 10		12.2260673	10.0121458	10.4152155	10.10649279		

```
## 11  0.7216344 -2.3708096  1.5806995 -1.05865986
## 12 -0.4599438 -1.2532586  0.9400402  0.30172019
## 13 -0.4816115 -0.1748480  1.1314302  0.08997319
## 14 -0.3492753  0.1085116 -0.2136959 -1.36339225
## 15 -0.5617660  0.5169205  0.9578389  0.28252387
## 16 -11.0128181 -9.3667726 -9.6848771 -7.81968877
## 17 -9.1664467 -9.3976351 -9.8660606 -9.62122272
## 18 -8.6459764 -10.0340337 -11.2405516 -11.06931283
## 19 -9.9499273 -10.7897064 -9.7435195 -10.37514498
## 20 -9.6631192 -10.4704356 -10.6740702 -10.79479772
```

Simulation of Classification Accuracy

Next, I will write a function to find classification accuracy for the `lda`, `knn`, and `tree` functions.

The function does a loop over replicates. In each replicate, it

- 1) Generates training and test data.
- 2) Fits the LDA, KNN, and Tree methods to the training data.
- 3) Finds the classification accuracy of each method on the test data.

It then summarizes the performance over replicates.

```
library(tree)
```

```
## Warning: package 'tree' was built under R version 4.5.2
```

```
library(class)
```

```
class_sim<-function(nrep,n1,n2,m1,m0,x11,x12,x21,x22,sd0)
{
```

```
  lda_acc=vector(length=nrep)
  knn_acc=vector(length=nrep)
  tree_acc=vector(length=nrep)
```

```
  #browser()
```

```
  for(r in 1:nrep)
```

```
  { #print(r)
```

```
    traindat=generateClassificationData(n1,n2,m1,m0,x11,x12,x21,x22,sd0,FALSE,TRUE)
    testdat=generateClassificationData(n1,n2,m1,m0,x11,x12,x21,x22,sd0,FALSE,TRUE)
```

```
    lda_mod=lda(traindat[,-1],grouping=traindat[,1])
```

```
    lda_acc[r]=sum(predict(lda_mod,testdat[,-1])$class==testdat[,1])/length(testdat[,1])
```

```
    bestk=find_k(traindat[,-1],traindat[,1],10,30)
```

```
    knn_acc[r]=sum(knn(traindat[,-1],testdat[,-1],k=bestk$best_k,c1=traindat[,1])==testdat[,1])/length(testdat[,1])
```

```

#note: it is necessary to specify model=T in the tree function to get cv.tree
#to work inside of a function. Otherwise, there is an error "unable to find
#object traindat" when running cv.tree. It is not clear to me why this is the case.
#It works outside of a the function with no problem. There is some issue with namespace
#and where tree is storing its information.
tree_mod=tree(v1~.,data=traindat,x=TRUE,y=TRUE,model=T)
cv_mod=cv.tree(tree_mod,FUN=prune.misclass)
best_size=cv_mod$size[which.min(cv_mod$dev)]
prune_tree=prune.tree(tree_mod,best=best_size)
tree.pred=predict(prune_tree,testdat,type="class")
tree_acc[r]=sum(tree.pred==testdat[,1])/length(testdat[,1])

}

par(mfrow=c(3,1))
hist(lda_acc,main="LDA classification accuracy",xlab="classification accuracy",probability = TRUE)
hist(knn_acc,main="KNN classification accuracy",xlab="classification accuracy",probability = TRUE)
hist(tree_acc,main="tree classification accuracy",xlab="classification accuracy",probability = TRUE)

return(list(lda=summary(lda_acc),knn=summary(knn_acc),tree=summary(tree_acc)))

#tree_mod=tree(traindat[,1]~traindat[,-1][[1]],traindat)
#tree_mod_cv=cv.tree(tree_mod)
#prune.tree(tree_mod)
#predict(tree_mod_cv,testdat[,,-1])

}

```

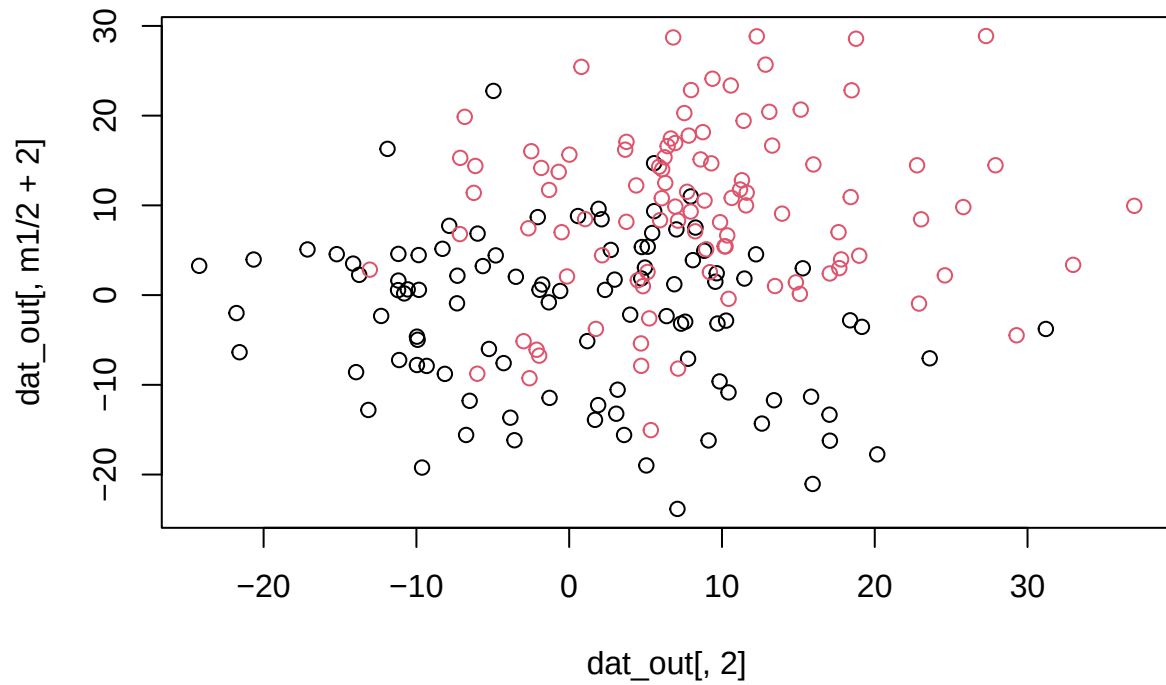
Next, we try some different simulation scenarios:

Scenario 1: Gaussian

We will start with a scenario favorable for LSD. Each class only has one component (i.e. $X_{11}=X_{12}$ and $X_{21}=x_{22}$), meaning that the data is Gaussian. I will start with a case with 20 informative variables and no noise variables:

Show a plot in 2D:

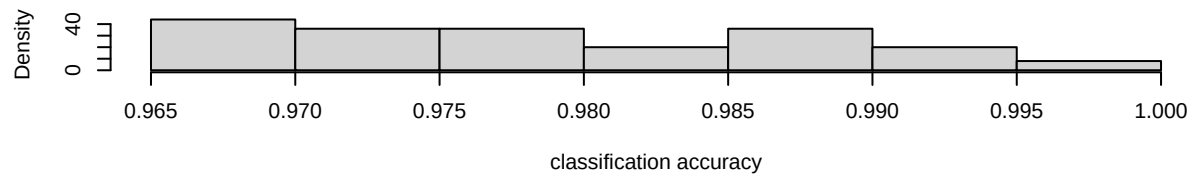
```
generateClassificationData(c(50,50),c(50,50),20,0,x11=c(0,0),x12=c(0,0),x21=c(10,10),x22=c(10,10),100,r
```



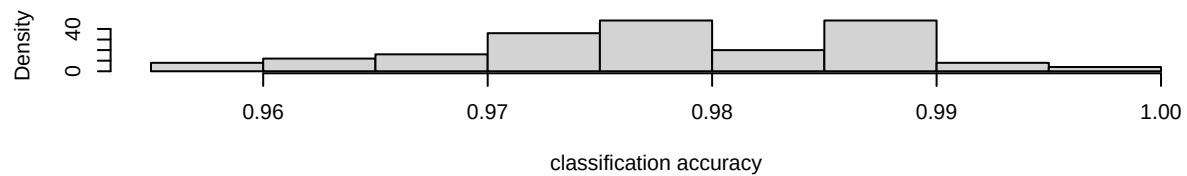
Run the simulation

```
class_sim(50,c(50,50),c(50,50),20,0,x11=c(0,0),x12=c(0,0),x21=c(10,10),x22=c(10,10),100)
```

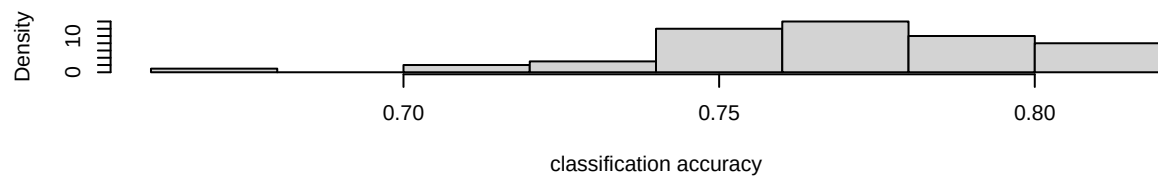
LDA classification accuracy



KNN classification accuracy



tree classification accuracy



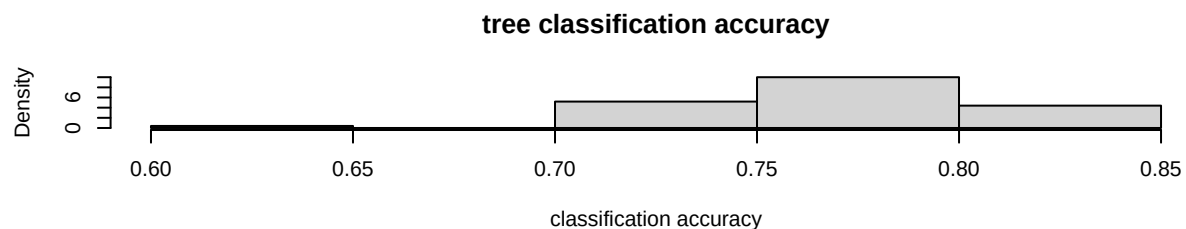
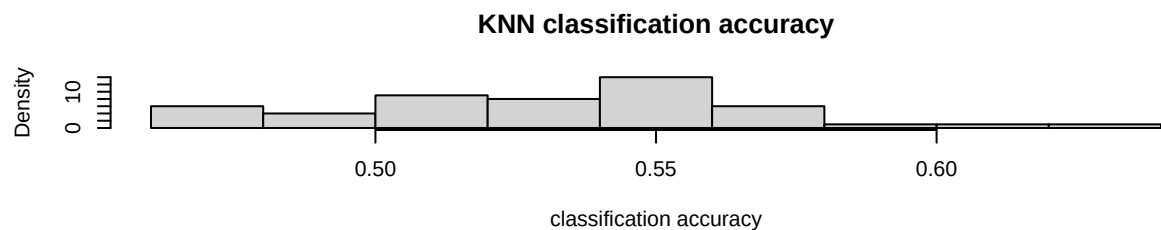
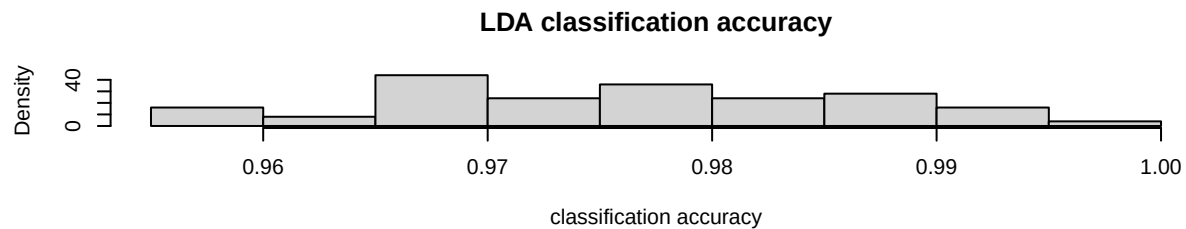
```
## $lda
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
## 0.9650 0.9750 0.9800 0.9811 0.9900 1.0000
##
## $knn
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.9550 0.9750 0.9800 0.9804 0.9900 1.0000
##
## $tree
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.6750 0.7562 0.7750 0.7726 0.7900 0.8200
```

LDA and KNN have high classification accuracy, in the upper 90% range. Tree classification does less well, with accuracy in the 70-80% range.

Now, I will add in 20 noise variables:

```
class_sim(50,c(50,50),c(50,50),20,20,x11=c(0,0),x12=c(0,0),x21=c(10,10),x22=c(10,10),100)
```

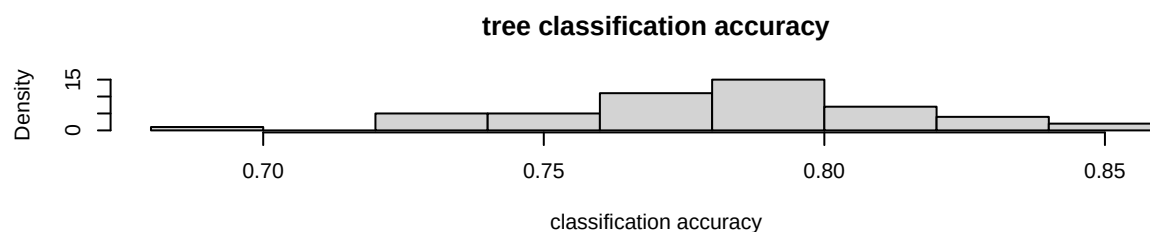
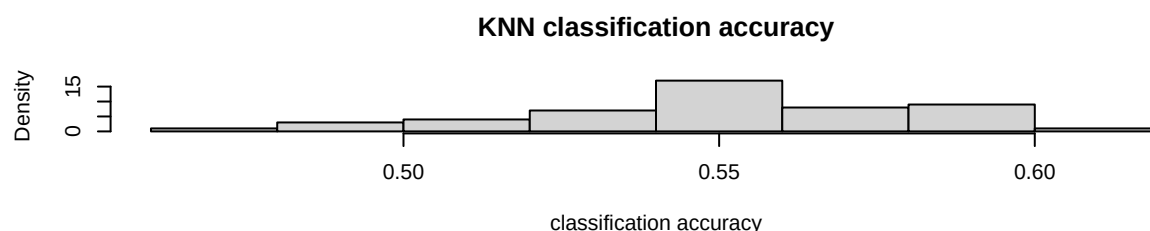
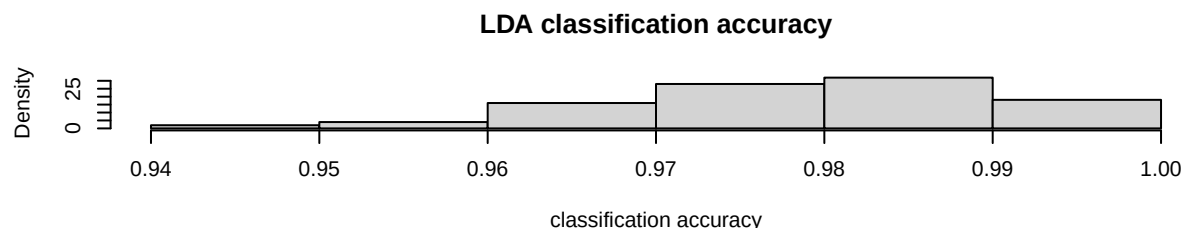


```
## $lda
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.9550 0.9700 0.9800 0.9784 0.9850 1.0000
##
## $knn
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.4600 0.5112 0.5400 0.5350 0.5600 0.6350
##
## $tree
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.6050 0.7500 0.7775 0.7755 0.8000 0.8350
```

LDA still does well, but KNN does poorly with classification accuracy in the 50% range. Tree classification does about the same as before, with accuracy in the 70-80% range. Keep in mind that since there are two classes of equal proportion, then a random guess would have a 50% chance of being correct. Thus, KNN is only doing a bit better than random chance.

Next, I reduce to 10 noise variables to see how KNN does:

```
class_sim(50,c(50,50),c(50,50),20,10,x11=c(0,0),x12=c(0,0),x21=c(10,10),x22=c(10,10),100)
```

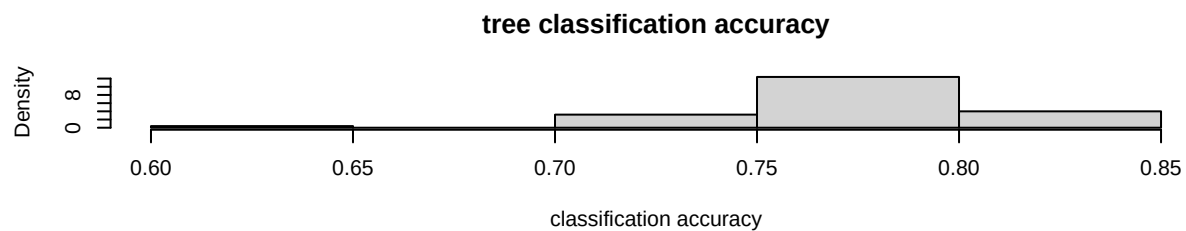
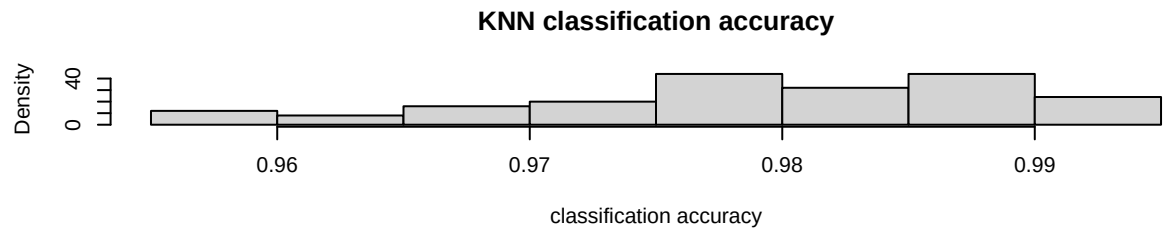
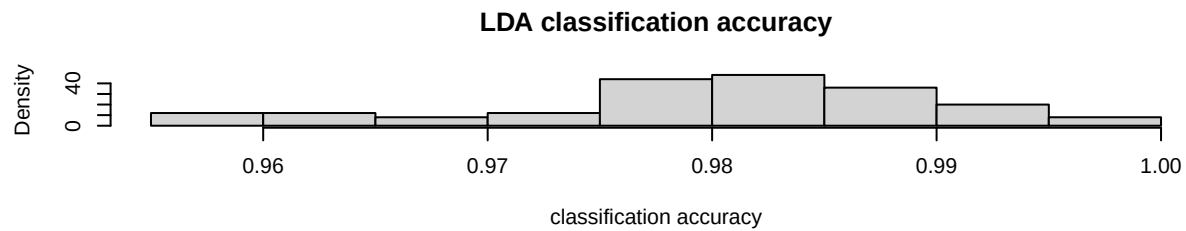


```
## $lda
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.9450 0.9750 0.9825 0.9809 0.9888 1.0000
##
## $knn
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.470 0.540 0.555 0.553 0.575 0.615
##
## $tree
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.6800 0.7650 0.7850 0.7837 0.8037 0.8550
```

KNN still does poorly.

Next try 5 noise variables

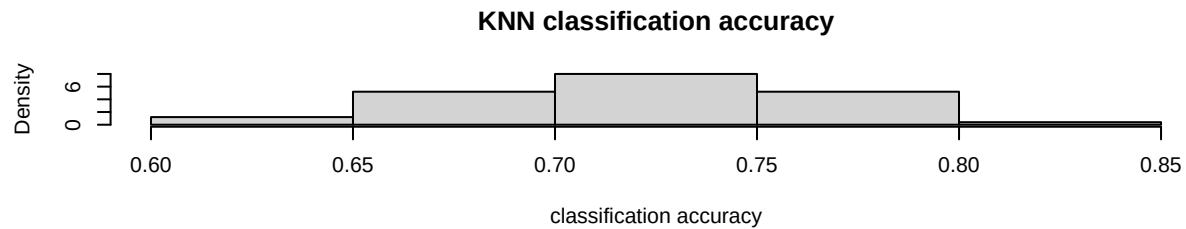
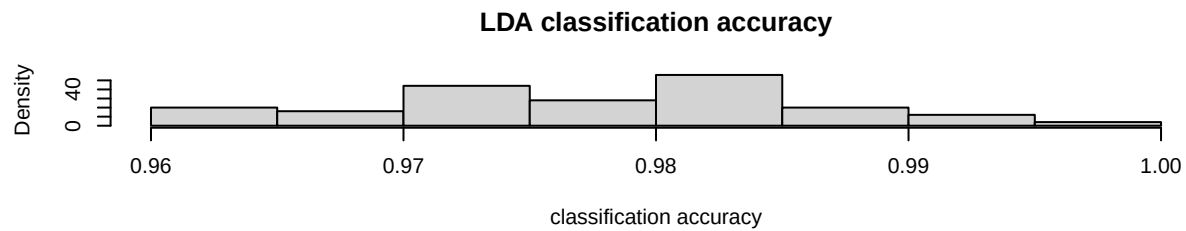
```
class_sim(50,c(50,50),c(50,50),20,0,x11=c(0,0),x12=c(0,0),x21=c(10,10),x22=c(10,10),100)
```



```
## $lda
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.9550 0.9800 0.9850 0.9822 0.9900 1.0000
##
## $knn
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.9550 0.9750 0.9825 0.9816 0.9900 0.9950
##
## $tree
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.6250 0.7650 0.7825 0.7790 0.8000 0.8450
```

and 3:

```
class_sim(50,c(50,50),c(50,50),20,3,x11=c(0,0),x12=c(0,0),x21=c(10,10),x22=c(10,10),100)
```

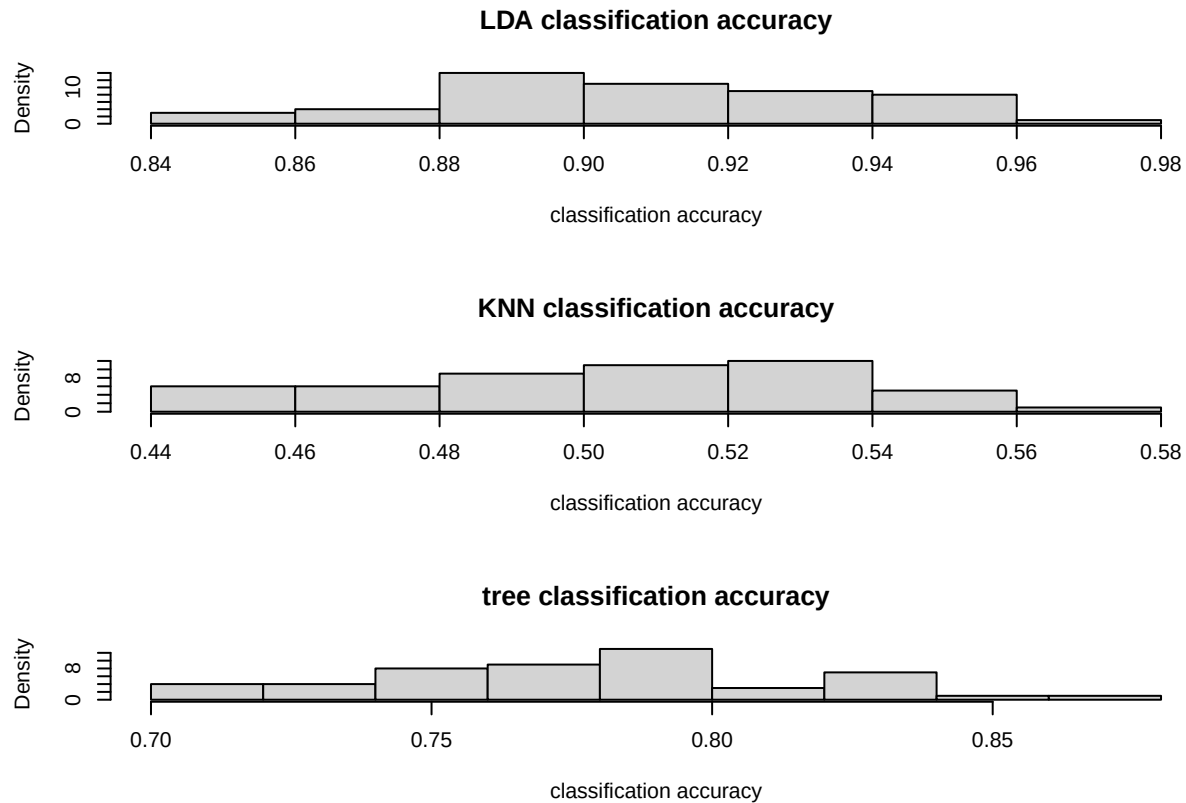


```
## $lda
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.9600 0.9750 0.9800 0.9799 0.9850 1.0000
##
## $knn
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.6250 0.6963 0.7300 0.7264 0.7550 0.8500
##
## $tree
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.6600 0.7562 0.7800 0.7780 0.8000 0.8400
```

Even with only three noise variables compared to 20 informative ones, KNN does much worse than LDA.

I will do one more scenario with many noise variables to see how LDA does:

```
class_sim(50,c(50,50),c(50,50),20,100,x11=c(0,0),x12=c(0,0),x21=c(10,10),x22=c(10,10),100)
```



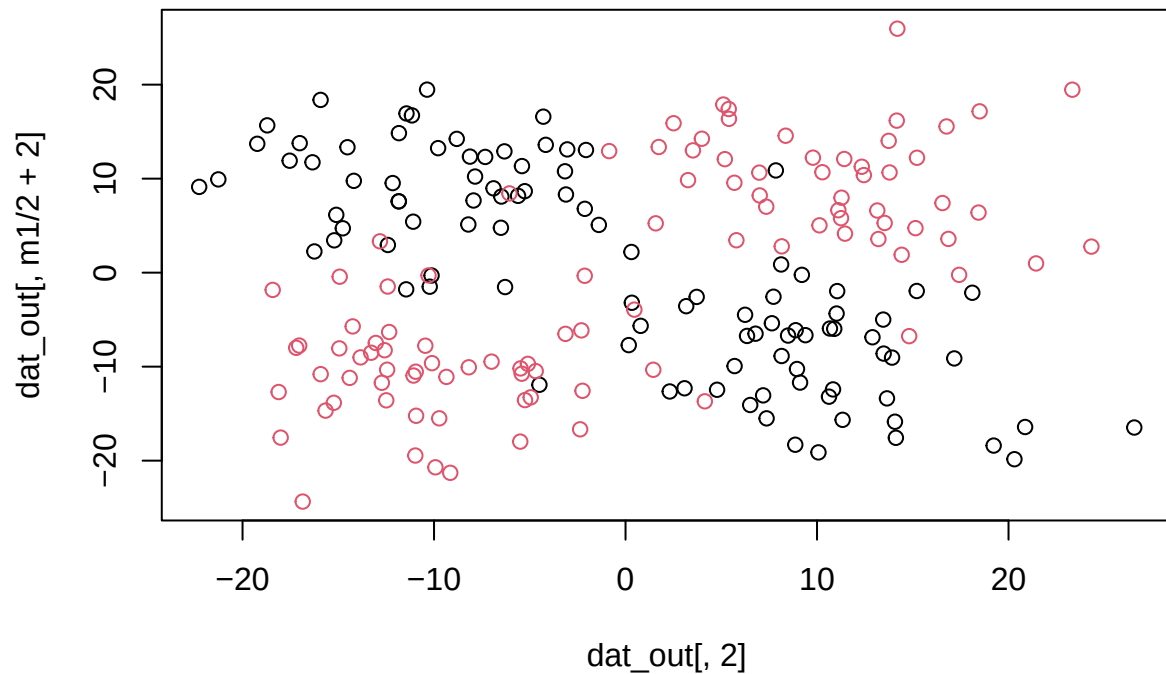
```
## $lda
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.8500 0.8900 0.9125 0.9107 0.9300 0.9700
##
## $knn
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.4400 0.4850 0.5075 0.5064 0.5300 0.5800
##
## $tree
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.7050 0.7512 0.7825 0.7801 0.8000 0.8650
```

We see that even with 5 times as many noise variables as informative ones, LDA achieves about 90% classification accuracy. Tree classification also is not much affected by the addition of noise variables, although it is less effective than LDA.

Scenario 2: Gaussian Mixture

Now, I will simulate more complex boundary by specifying two components for each class:

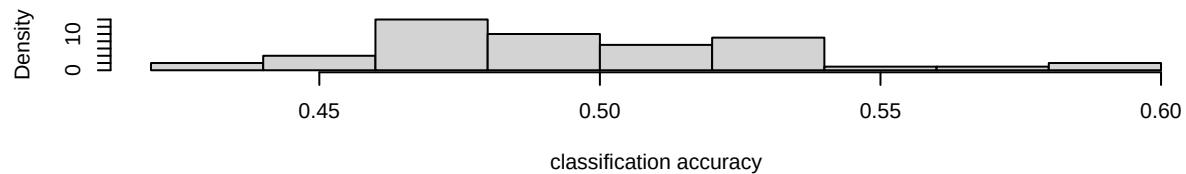
```
generateClassificationData(c(50,50),c(50,50),20,0,x11=c(-10,10),x12=c(10,-10),x21=c(-10,-10),x22=c(10,10))
```



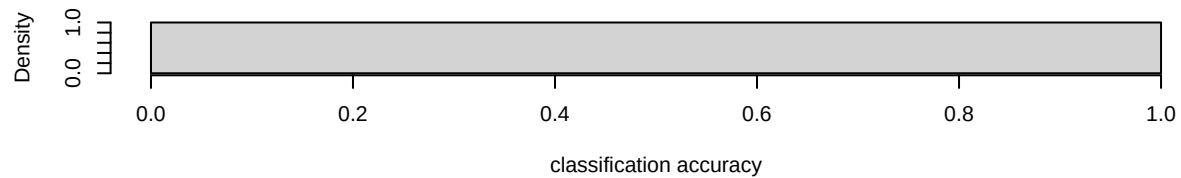
I will first start with 20 informative variables and no noise variables:

```
class_sim(50,c(50,50),c(50,50),20,0,x11=c(-10,10),x12=c(10,-10),x21=c(-10,-10),x22=c(10,10),30)
```

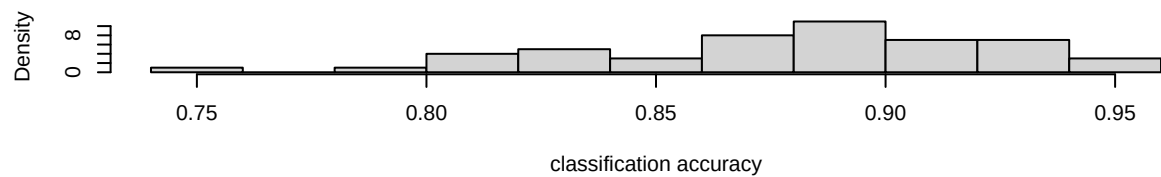
LDA classification accuracy



KNN classification accuracy



tree classification accuracy



```
## $lda
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
```

```
## 0.4250 0.4713 0.4950 0.4971 0.5225 0.5950
##
## $knn
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##         1         1         1         1         1         1
##
## $tree
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.7550 0.8600 0.8875 0.8823 0.9187 0.9500
```

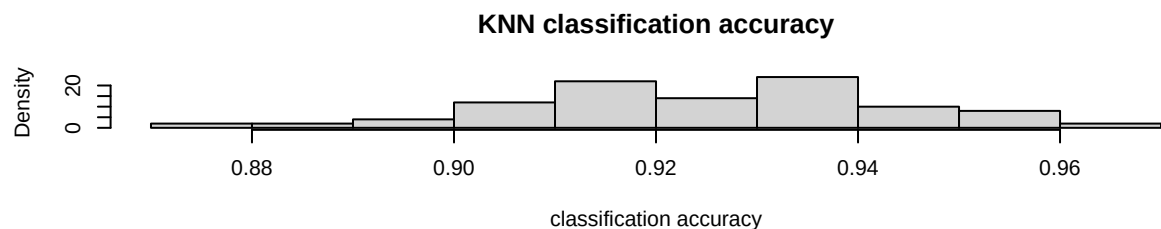
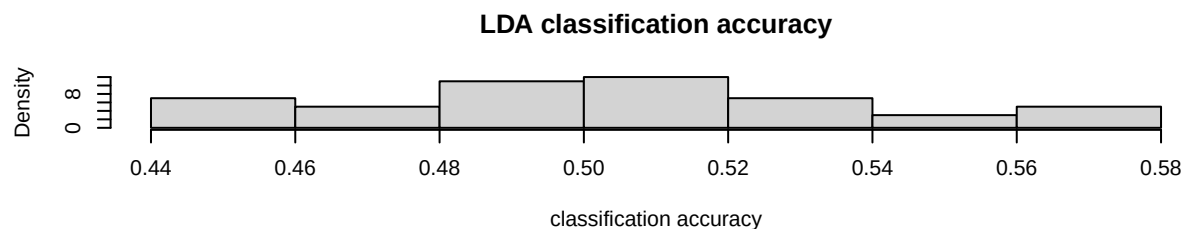
LDA does worse, achieving only about 50% classification accuracy - no better than random chance. LDA does poorly with the complex boundary. LDA assumes a Gaussian distribution for the predictors. This assumption is met above and LDA does well. Here, LDA does very poorly.

In contrast, KNN does extremely well, achieving 100% classification accuracy in all 50 replicates. KNN can handle a more complicated structure to the class boundaries. Note that the standard deviation of the noise was reduced to 30 for this scenario in contrast to 100 in the previous scenario. This is why KNN performs better here than in the previous scenario with no noise variables.

Tree classification also does well, achieving 85-90% accuracy.

Next, I will add in noise variables:

```
class_sim(50,c(50,50),c(50,50),20,10,x11=c(-10,10),x12=c(10,-10),x21=c(-10,-10),x22=c(10,10),30)
```

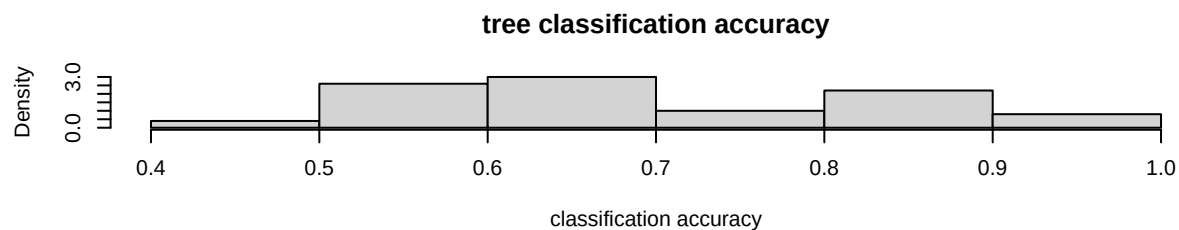
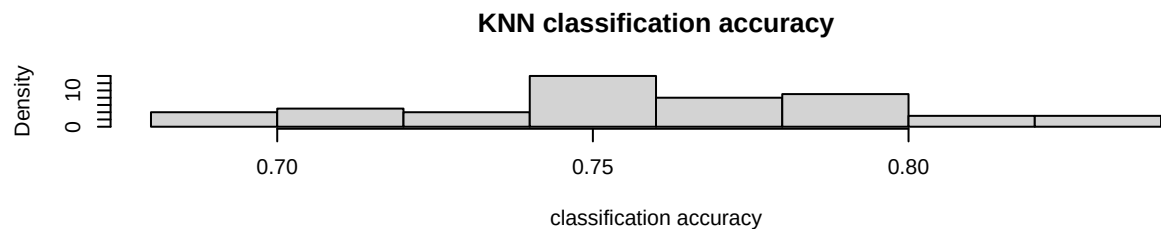
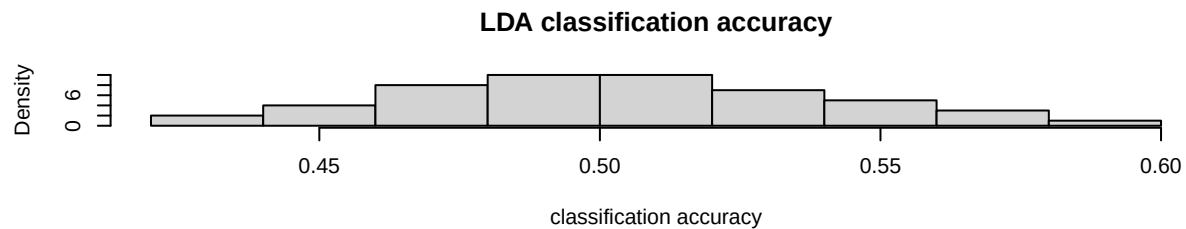


```
## $lda
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.4450 0.4850 0.5075 0.5064 0.5300 0.5750
##
```

```
## $knn
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.8750 0.9150 0.9275 0.9274 0.9400 0.9650
##
## $tree
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.5000 0.7000 0.8050 0.7741 0.8625 0.9600
```

With 10 noise variables and 20 informative ones, the accuracy of KNN is reduced but is still above 90%. The accuracy of tree classification is reduced to the 70-85% range.

```
class_sim(50,c(50,50),c(50,50),20,20,x11=c(-10,10),x12=c(10,-10),x21=c(-10,-10),x22=c(10,10),30)
```



```
## $lda
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.4250 0.4800 0.5075 0.5067 0.5337 0.5900
##
## $knn
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.6900 0.7412 0.7600 0.7604 0.7900 0.8350
##
## $tree
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.4400 0.5950 0.6700 0.6995 0.8287 0.9550
```

With 20 noise variables, the accuracy of KNN is down to the 70% range. For both scenarios, LDA is barely better than chance. Tree classification does worse with 63-78% IQR for accuracy.

Summary

Comparing LDA, KNN, and Tree classification we conclude that 1) LDA is much more robust to the presence of extraneous noise variables than KNN, which is very sensitive to them. 2) KNN is more robust to data where the data is not Gaussian. LDA performs poorly with more complicated class boundaries. At least in our example, KNN was able to perform well. Tree classification was in the middle. It did better than KNN but worse than LDA in the first scenario with Gaussian data and a lot of noise. It did better than LDA but worse than KNN in the second scenario with more complicated boundaries but less noise.